**Course Code: CSA1741 Course Name: ARTIFICIAL INTELLIGENCE**
**Slot: B**

# ASSIGNMENT 1 – CO1

**Nature of Assignment: Individual** **Duration: 1 Week** **Total Marks: 50**

**Student Name: S. Arshad Ali** **Reg. No:192412204**

**CO1:** Foundation and history of AI, intelligent agents (agents and environments, rationality, nature of environments, structure of agents), problem-solving agents, problem formulation, searching for solutions, uninformed search strategies and search with partial information.

**Assignment Title:** Design, Analyse and Validate an Efficient Search Strategy for a Hospital Medicine-Delivery Robot

## Scenario — Hospital Medicine-Delivery Robot

A multi-block hospital uses an autonomous robot to carry medicines from the central pharmacy to wards. The building is modelled as a graph of locations (junctions, corridors, lifts, ward doors). The robot may move between connected locations and every move has a travel-time cost.

As new wings are added, the state space of the robot may grow:

1,000 → 10,000 → 100,000 → 1,000,000+ states

You are required to investigate, design and compare uninformed search strategies by which the robot finds a route from the pharmacy to a requested ward as the hospital grows.

Your solution must remain practical for both small and very large state spaces. Any preprocessing (for example, building adjacency lists or a duplicate-detection structure) must be included in your efficiency analysis.

**Design Challenge (Day 4):** The robot's localisation sensor now fails intermittently, so the robot no longer knows exactly where it is. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

### TASK 1 – Design the Agent and the Search Algorithms (5 Marks)

Before designing algorithms, define the agent and the problem:

- Specify the PEAS description of the robot and classify its environment (observable, deterministic, episodic/sequential, static/dynamic, discrete/continuous, single/multi-agent), justifying each choice
- State which view of AI (acting rationally, thinking rationally, acting humanly, thinking humanly) the robot follows and why it is a rational problem-solving agent. Relate the problem type briefly to the early history of AI problem solving
- Formulate the problem: states, initial state, actions, transition model, goal test, path cost

Then design and propose at least **three meaningfully different** uninformed search strategies for finding a route. Iterative and recursive implementations of the same underlying strategy will not be considered two different approaches.

Your proposed approaches must include:

- At least one iterative approach (queue- or stack-based)
- At least one recursive approach
- A clear explanation of the basic idea behind each approach
- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required, such as adjacency-list construction or an explored set

You are expected to select the approaches independently. The appropriateness and diversity of the selected strategies are part of the assessment.

**Evidence Expected:** PEAS and environment classification, complete problem formulation, three distinct strategies with pseudocode/flowcharts, preprocessing requirements and justification for each.

**TASK 2 – Identify the Basic Operation (2 Marks)**

For each proposed algorithm, identify the basic operation that predominantly determines its execution time.

Discuss:

- What the basic operation is (for example, node expansion, successor generation, goal test or frontier insertion)
- Where it occurs in the algorithm

- Why it is appropriate for efficiency analysis

- How the number of times it executes may change as the state-space size increases

Do not simply name an operation. Relate it to the behaviour of the algorithm.

**Evidence Expected:** Correct identification and justification of the basic operation for each algorithm.

**TASK 3 – Derive the Time- and Space-Complexity Functions Mathematically (6 Marks)**

Derive the time and space complexity for each proposed approach using mathematical analysis.

Discuss:

- The input size (branching factor $b$, shallowest-goal depth $d$, maximum depth $m$, optimal cost $C^*$, minimum step cost $\varepsilon$)
- The number of times the basic operation executes, derived by summing the nodes at each level
- The mathematical expression for time and for memory (frontier size)
- Any preprocessing cost involved
- The overall cost when preprocessing and searching are considered together

You must show the mathematical reasoning. Merely stating that an algorithm is O(b^d) will not receive full credit.

**Evidence Expected:** A clear derivation of time and space functions for each approach, including operation counts and preprocessing costs.

**TASK 4 – Analyse Best, Average and Worst-Case Behaviour (4 Marks)**

Analyse each algorithm under different input conditions.

Discuss, wherever applicable:

- What input condition produces the best case (for example, the goal lies on the first branch explored)
- What condition produces the average case
- What condition produces the worst case (for example, goal at maximum depth, goal unreachable, cheap-edge detours)

- How the number of nodes generated changes in each case, and why

Do not simply provide a table of complexity values. Explain the conditions that lead to each case.

**Evidence Expected:** A justified best-, average- and worst-case analysis supported by mathematical reasoning.

**TASK 5 – Perform Asymptotic Analysis (3 Marks)**

Express and compare the growth behaviour of the algorithms using O, Ω and/or Θ.

Discuss:

- How each algorithm behaves as the state space becomes very large
- How the growth rates of the approaches differ in time and in memory
- What the growth means when the hospital grows from 1,000 to 1,000,000+ states, for example, whether memory or time becomes the limiting factor first

Interpret the behaviour in the context of the hospital robot rather than merely stating the notation.

**Evidence Expected:** Correct asymptotic representation and a meaningful comparison of growth behaviour.

**TASK 6 – Formulate the Recurrence Relation (3 Marks)**

For the recursive approach, formulate a recurrence relation directly from the behaviour of your algorithm.

Identify clearly:

- The size of the original problem (depth limit or remaining depth)
- The size of the recursive subproblem
- The number of recursive calls (related to the branching factor)
- The work performed outside the recursive calls
- The base case
- How these components lead to the recurrence

The recurrence must be derived from your algorithm, not copied from a textbook.

**Evidence Expected:** A correct recurrence with clear identification of its components and base case.

**TASK 7 – Solve the Recurrence Relation (4 Marks)**

Solve the recurrence from Task 6 using an appropriate analytical method (substitution, iteration, recursion tree or another method).

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the recursive algorithm as depth or branching factor increases
- How the result extends to iterative deepening, where the recursive search is repeated for increasing limits

Show the analytical process, not only the final result.

**Evidence Expected:** A complete solution with justification of the method and interpretation of the result.

**TASK 8 – Implement and Test the Algorithms (4 Marks)**

Implement all proposed algorithms in Python and test them experimentally.

Use generated hospital graphs of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 states

Ensure that:

- All algorithms return correct routes
- The same or equivalent graphs and start/goal pairs are used for comparison
- Testing conditions are consistent and the random seeds are reported
- Test cases include a reachable goal, an unreachable goal and a start equal to the goal
- The implementation corresponds to the algorithms analysed in Tasks 1–7

The purpose of implementation is to generate evidence for validating your theoretical analysis.

**Evidence Expected:** Working code, test inputs, expected behaviour and actual outputs, with a reproducible testing procedure.

**TASK 9 – Record Experimental Performance (3 Marks)**

Collect and present performance data for each algorithm across the different state-space sizes.

Record, with code and outputs as proof:

- Nodes generated and nodes expanded
- Maximum frontier size (memory proxy)
- Solution depth and path cost, and whether the path is optimal
- Execution time (averaged over at least 5 runs)

Present the results using tables and graphs (use a log scale where appropriate) so that approaches can be compared easily.

**Evidence Expected:** Systematically collected experimental data with clear tables and graphs.

**TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)**

Compare your theoretical analysis with the experimental results.

Discuss:

- Whether the observed growth of nodes generated and of frontier size agrees with the theoretical complexity
- Which algorithm scales better as the state space increases
- Any differences between theory and observation, and the possible reasons (graph structure, duplicates, tie-breaking, language overhead)

Interpret the results rather than simply reproducing tables or graphs.

**Evidence Expected:** A reasoned comparison between theoretical and observed performance.

**TASK 11 – Recommend and Defend the Final Design (4 Marks)**

Based on your mathematical analysis and experimental findings, recommend the most appropriate search strategy for the hospital robot.

Discuss:

- Which approach you recommend and why it is more appropriate than the alternatives

- Its theoretical efficiency and experimentally observed performance
- Any preprocessing cost, trade-offs or limitations
- Whether it remains practical as the hospital grows

Your recommendation must be based on your own evidence. Simply choosing the algorithm with the lowest Big-O complexity is not sufficient.

**Evidence Expected:** A technically justified recommendation supported by analysis and experiment.

**DAY-4 DESIGN CHALLENGE**

**TASK 12 – Re-evaluate and Adapt the Design (8 Marks)**

**Changed Requirement**

After a power restart or sensor fault, the robot no longer knows exactly where it is. It knows only that it is at one of $k$ possible junctions. It must still reach the pharmacy docking station, whichever junction it is actually at.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

**A. Impact Analysis – 2 Marks**

- What has changed from the original problem and which original assumptions (fully observable, known start state) are affected?
- Why does the robot's knowledge now form part of the state?

**B. Evaluation of the Original Design – 2 Marks**

- Can your original algorithm still be applied? What does a "state" become?
- How large can the new search space be for $k$ possible locations, and is the original design acceptable?

**C. Adaptation or Redesign – 2 Marks**

- Describe the revised strategy (for example, search over belief states, or search with local sensing and a conditional plan), what has changed and the effect on complexity
- If you retain the original solution, give sufficient technical justification

**D. Evidence-Based Defence – 2 Marks**

- Defend your decision using mathematical analysis and a small experiment (belief states generated and plan length for $k$ = 2, 4, 8, with and without a local sensor), with code and output as proof.

**MARK DISTRIBUTION – ASSIGNMENT_1**

| Task | Marks |
|---|---|
| Task 1 – Agent and Algorithm Design | 5 |
| Task 2 – Basic Operation | 2 |
| Task 3 – Mathematical Complexity Analysis | 6 |
| Task 4 – Best/Average/Worst-Case Analysis | 4 |
| Task 5 – Asymptotic Analysis | 3 |
| Task 6 – Recurrence Formulation | 3 |
| Task 7 – Recurrence Solution | 4 |
| Task 8 – Implementation & Testing | 4 |
| Task 9 – Experimental Performance | 3 |
| Task 10 – Theory vs Experiment | 4 |
| Task 11 – Final Design Recommendation | 4 |
| **Subtotal** | **42** |
| Task 12 – Day-4 Design Challenge | 8 |
| **TOTAL** | **50** |

**Abstract**

This assignment designs and evaluates uninformed search strategies for an autonomous hospital medicine-delivery robot. The hospital is represented as a weighted graph whose vertices are junctions, corridors, lifts and ward doors. Breadth-First Search (BFS), Uniform Cost Search (UCS) and Iterative Deepening Depth-First Search (IDDFS) are selected because they differ in search order, solution guarantee and memory behaviour. Their basic operations and mathematical costs are derived, and a Python program is supplied for reproducible graph generation, correctness tests and benchmarking. The Day-4 localisation failure is handled by changing the state from a single location to a belief set of possible locations and by planning with sensing.

**Assumptions and notation**

The graph is G=(V,E). N=|V| is the number of locations; b is the maximum/average branching factor; d is the depth of the shallowest goal; m is the maximum reachable depth; C* is the optimal path cost; ε is the minimum positive edge cost. Edges have positive travel-time costs. The graph is finite for each experiment. Duplicate states are detected for BFS and UCS. IDDFS uses path-based cycle checking; it may revisit a state in different iterations.

Complexity formulae below use the usual tree-search upper bounds unless explicitly described as graph-search. In a graph with N vertices and E edges, adjacency-list preprocessing is Θ(N+E), and a search that fully explores the graph is O(N+E), except that IDDFS can repeat expansions across depth limits.

**TASK 1 — Design the agent and search algorithms**

**1.1 PEAS description**

| Component | Description |
|---|---|
| Performance measure | Correct medicine and ward; minimum travel time; on-time delivery; collision avoidance; safe lift use; low energy; reliable communication; no wrong-patient delivery. |
| Environment | Hospital blocks, corridors, junctions, lifts, pharmacy, ward doors, people, carts and temporary closures. |
| Actuators | Drive wheels, steering, brakes, lift interface, medicine compartment lock, display/speaker, wireless communication |
| Sensors | Wheel odometry, localisation, lidar/depth camera, proximity sensors, RFID/QR ward markers, lift status, map and task database |

**1.2 Environment classification**

Partially observable in real operation: sensors do not reveal every corridor condition or exact location continuously. The original planning model assumes a known map and known start location (effectively fully observable for route planning).

Mostly deterministic: a planned move has a known travel-time cost, although crowds, doors and lift delays introduce uncertainty in practice.

Sequential: every move changes the robot's location and affects later decisions.

Dynamic: people, temporary obstacles, lift status and corridor access may change while the robot is moving.

Discrete for the search model: graph nodes and actions are discrete. Physical position, time and sensor readings are continuous.

Single-agent for route planning: the robot is the decision-making agent. Humans and other robots are treated as dynamic environment elements, not cooperative search agents.

**1.3 AI view and rationality**

The robot follows the acting-rationally view. It chooses actions that maximize expected performance given its percepts, map, costs and knowledge. It is rational because it selects a route according to the defined performance measure (safety and delivery first, then travel-time cost), rather than merely imitating a human. Classical AI problem solving in the 1950s–1960s formalised tasks as state spaces and used symbolic search; early work such as Newell and Simon's problem-solving systems and the General Problem Solver helped establish this approach.

**1.4 Problem formulation**

States: a hospital location $v \in V$ (original, fully localised problem).

Initial state: pharmacy location $s \in V$.

Actions: move along any outgoing corridor/connection $(v,u) \in E$; optionally use lift edges.

Transition model: Result(v, move(v,u))=u.

Goal test: v=g, where g is the requested ward door.

Path cost: sum of edge travel times, $\text{Cost}(\text{path}) = \Sigma\, c(v_i, v_{i+1})$. All costs are positive.

**1.5 Selected uninformed search strategies**

**A. Breadth-First Search (BFS) — iterative queue**

BFS expands locations in nondecreasing number of edges from the start. It is complete for finite graphs and returns a minimum-edge route. It is cost-optimal only if every edge has the same cost. It is selected as a simple baseline and performs well when corridor steps are uniform.

BFS(G, s, g):
Q ← FIFO queue containing s

```
parent[s] ← NIL; discovered ← {s}
while Q is not empty:
u ← Q.pop_front()
if Goal(u): return Reconstruct(parent, u)
for each v in Adj[u]:
if v not in discovered:
discovered.add(v); parent[v] ← u
Q.push_back(v)
return FAILURE
```

Preprocessing: adjacency lists; discovered set; parent map for route reconstruction.

**B. Uniform Cost Search (UCS) — iterative priority queue**

UCS expands the frontier node with the smallest accumulated travel cost g(n). With positive edge costs and proper relaxation/duplicate handling, UCS is complete and optimal. It is selected as the principal algorithm because hospital travel-time edges are not equal.

```
UCS(G, s, g):
PQ ← min-priority queue containing (0, s)
best[s] ← 0; parent[s] ← NIL
while PQ is not empty:
(cost, u) ← PQ.pop_min()
if cost ≠ best[u]: continue // stale entry
if Goal(u): return Reconstruct(parent, u)
for each (v, w) in Adj[u]:
new ← cost + w
if new < best[v]:
best[v] ← new; parent[v] ← u
PQ.push(new, v)
return FAILURE
```

Preprocessing: adjacency lists, best-cost dictionary, parent map and min-priority queue. Heap operations cost O(log F), where F is frontier size.

**C. Iterative Deepening Depth-First Search (IDDFS) — recursive depth-limited search**

IDDFS performs depth-limited DFS repeatedly with limits 0,1,2,… until a goal is found. It is complete for finite branching and finds a shallowest goal (edge-count optimal), while using memory proportional to search depth. It is selected to demonstrate recursive search and low-memory behaviour. It is not travel-time optimal when edge costs differ.

```
IDDFS(G, s, g):
for limit = 0, 1, 2, ...:
result ← DLS(s, limit, path={s})
if result ≠ CUTOFF: return result
```

DLS(u, limit, path):

if Goal(u): return path
if depth(path) = limit: return CUTOFF
for v in Adj[u]:
if v not in path: // path-cycle check
result ← DLS(v, limit, path ∪ {v})
if result is a solution: return result
return CUTOFF

Preprocessing: adjacency lists. The recursion stack and current-path set prevent cycles along the active path; unlike BFS/UCS, a global explored set cannot be naively reused across depth limits.

**TASK 2 — Basic operation**

BFS: successor examination / discovery test, performed inside `for v in Adj[u]`. Each examined edge can cause a membership test and at most one enqueue. It dominates runtime as vertices and corridors grow.

UCS: priority-queue insertion/removal and relaxation of edges, inside the successor loop. Each successful relaxation inserts a candidate; heap operations add a logarithmic factor. This reflects both graph connectivity and frontier ordering.

IDDFS: recursive node expansion (successor generation) inside DLS. The same shallow nodes are expanded repeatedly at increasing limits, so operation count depends on d and branching factor, not only unique vertices.

**TASK 3 — Mathematical time and space derivation**

**3.1 BFS**

At level i, a tree search can contain at most b^i nodes. If the shallowest goal is at depth d, the number of nodes generated through depth d is

$T_{BFS}(d)=\Sigma(i=0 \text{ to } d)\ b^i = (b^{(d+1)}-1)/(b-1)$, for $b>1$; for $b=1$, $T=d+1$.

Each generated node is inserted once in a tree. Thus node-operation count is $\Theta(\Sigma 0^d\ b^i)=\Theta(b^d)$ for $b>1$. With adjacency lists, edge examinations are $O(b\cdot T)=O(b^{(d+1)})$; the conventional time bound is $O(b^{(d+1)})$ (often written $O(b^d)$ when one level factor is suppressed).

BFS frontier contains the current level and the next level; worst-case memory is $\Theta(b^d)$ (more precisely $O(b^d)$ nodes). Graph-search duplicate detection may use $O(N)$ additional memory. Including preprocessing: $T_{total}=\Theta(N+E)+O(b^{(d+1)})$; $S_{total}=O(N)+O(b^d)$.

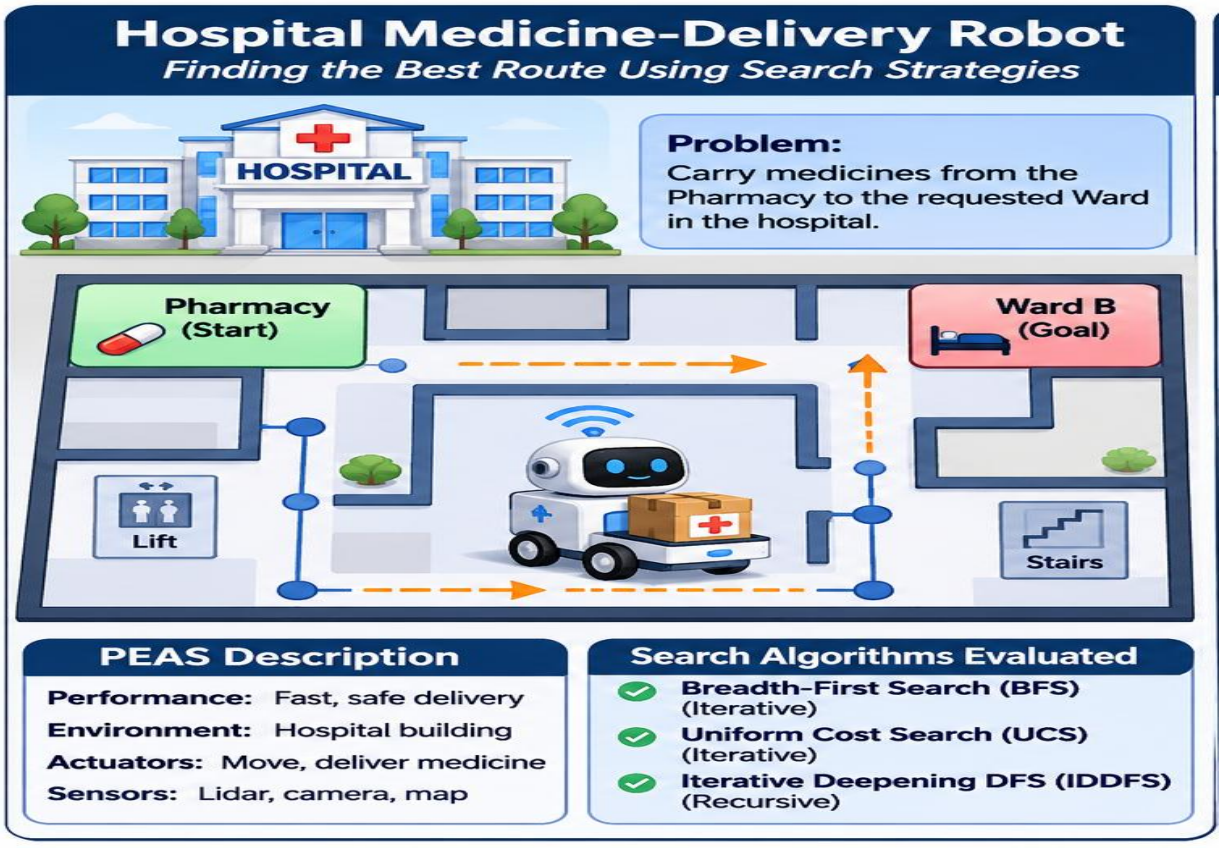
**3.2 UCS**

UCS orders by accumulated cost. With minimum step cost $\varepsilon>0$, a node with path cost $\le C^*$ can occur at depth at most $\text{floor}(C^*/\varepsilon)$. Let $D=\lfloor C^*/\varepsilon \rfloor$. In the worst-case tree, nodes generated before the optimal goal are bounded by

$$T_{UCS} \le \sum_{i=0}^{D} b^i = \frac{b^{D+1}-1}{b-1} = O(b^{\lfloor C^*/\varepsilon \rfloor+1}).$$

Each successor edge is examined when a node is expanded; a successful relaxation performs a heap insertion $O(\log F)$, and each pop costs $O(\log F)$. Therefore a more explicit bound is $O((V_g+E_g)\log F)$, where $V_g, E_g$ are the explored subgraph and $F \le O(b^D)$ is the maximum frontier. The tree upper bound is $O(b^{D+1}\log(b^D))$.

Space: best-cost and parent maps $O(V_g)$, plus heap frontier $O(b^D)$ in the tree bound. Total including preprocessing: $\Theta(N+E)$+search; memory $O(N+E)$ for the graph plus $O(V_g+F)$.




### 3.3 IDDFS

A depth-limited DFS with limit $\ell$ expands nodes up to depth $\ell$. Repeating limits $0 \ldots d$ means a node at level $i$ is regenerated in every iteration with limit $i, i+1, \ldots, d$, hence $d-i+1$ times. The total node count is

$$T_{IDDFS}(d)=\sum_{i=0}^{d} (d-i+1)b^i$$

$$= b^d \sum_{j=0}^{d} (j+1)/b^j, \text{ after setting } j=d-i.$$

For $b>1$ the geometric-weighted sum converges to $b/(b-1)^2$ as $d$ grows, so $T_{IDDFS}=\Theta(b^d)$, with a larger constant than BFS. It is commonly bounded by $O(b^d)$.

Space: recursion stack and path set O(d); a depth-first frontier is O(bd) if storing all successors at each level, or O(bd) conventional tree-search memory. In this implementation, adjacency lists O(N+E) are shared; search auxiliary memory is O(d) along the active path (excluding Python call overhead). Total: Θ(N+E)+Θ(b^d) time and Θ(N+E)+O(d) auxiliary memory.

### 3.4 Input-size and preprocessing discussion

N alone does not determine search cost: topology, branching factor, goal depth, edge weights and duplicate paths matter. Constructing adjacency lists from an edge list costs Θ(N+E) time and space. A hash-based explored/best-cost structure has expected O(1) membership/update; a heap adds O(log F). For a graph with N=10^6, storing the graph may itself be memory-intensive; the robot should not repeatedly rebuild it for each delivery.

### TASK 4 — Best, average and worst cases

| Algorithm | Best case | Average case | Worst case |
|---|---|---|---|
| BFS | Goal is start: Θ(1). Or goal is first node in first layer: small constant expansions. | Depends on goal position and layer widths; with uniform random goal in a regular tree, expected explored nodes are sum of preceding layers plus a fraction of goal layer. | Goal at depth d after all nodes at levels <d and most nodes at d are processed; unreachable goal explores all reachable vertices. Tree bound Θ(b^d) nodes, frontier Θ(b^d). |
| UCS | Start=goal: Θ(1). Otherwise goal is uniquely cheapest and popped early. | Depends on cost distribution and number of states with g(n)<C*. If many cheap detours exist, they are expanded first. | Many paths have cost below C*, or goal unreachable. Bound via D=⌊C*/ε⌋: exponential tree exploration; heap overhead O(log F). |
| IDDFS | Start=goal at limit 0: Θ(1). | Goal depth d and branching varies; nodes near depth d are repeated fewer times than root-level nodes. | Goal is last at depth d or absent. Repeated shallow expansions; Θ(b^d) nodes, O(d) auxiliary stack/path memory. |

Average-case values cannot be uniquely specified without a probability distribution over hospital graphs, goal locations and edge weights. Therefore the experiment fixes a seed and

graph-generation model and reports measured means over five runs. This is more defensible than claiming a universal average-case formula.

**TASK 5 — Asymptotic analysis**

For tree-like search with b>1:

BFS: T=O(b^(d+1)) and Ω(1); under a full level-wise exploration T=Θ(b^d). S=Θ(b^d).

UCS: T=O(b^(⌊C*/ε⌋+1 log F); exponential in C*/ε in the worst case. S=O(b^(C*/ε)) plus graph and cost maps.

IDDFS: T=Θ(b^d) in the standard bounded-branching tree model; auxiliary search memory O(d), much smaller than BFS/UCS frontier memory.

As the hospital grows from 10^3 to 10^6 locations, memory may become the first practical limit for BFS and UCS because they retain large frontiers and explored/cost dictionaries. IDDFS retains only a depth-sized path but may spend much more time revisiting nodes. In a finite sparse graph, graph-search BFS/UCS are O(N+E) in the explored graph (UCS with heap factors); the exponential tree bounds are worst-case search-space bounds, not a prediction that every hospital graph is exponential.

**TASK 6 — Recurrence relation for recursive DLS / IDDFS**

Let R(ℓ) be the number of calls/expansions of recursive depth-limited search with remaining depth ℓ in a full b-ary tree. At ℓ=0, the node is checked but no child call is made: R(0)=1. For ℓ>0, one current call performs constant local work (goal/depth checks and loop setup) and invokes b subproblems of size ℓ−1:

R(ℓ)=bR(ℓ−1)+1, R(0)=1.

If counting edge/successor tests, the local work is Θ(b), giving R(ℓ)=bR(ℓ−1)+Θ(b). Both recurrences have the same asymptotic solution for b>1. IDDFS repeats DLS at limits 0 through d, so total recurrence-derived calls are I(d)=Σ(ℓ=0 to d)R(ℓ).

**TASK 7 — Solve the recurrence**

Use iteration (substitution), applicable because each level replaces R(ℓ−1) with the same multiplicative factor b and additive constant:

R(ℓ) = bR(ℓ−1) + 1
= b[bR(ℓ−2)+1] + 1
= b²R(ℓ−2) + b + 1
= b³R(ℓ−3) + b² + b + 1
...
= b^ℓ R(0) + Σ(j=0 to ℓ−1) b^j
= b^ℓ + (b^ℓ−1)/(b−1)
= (b^(ℓ+1)−1)/(b−1), b>1
Therefore R(ℓ)=Θ(b^ℓ).

For IDDFS:

I(d)=Σ(ℓ=0 to d) (b^(ℓ+1)−1)/(b−1)

$= [b/(b-1)] \cdot [(b^{(d+1)}-1)/(b-1)] - (d+1)/(b-1)$

$= \Theta(b^d)$, for $b>1$.

Thus recursive DLS is exponential in limit depth for a full branching tree, while IDDFS remains asymptotically $\Theta(b^d)$ but repeats expansions. The benefit is $O(d)$ active-stack memory rather than BFS's $\Theta(b^d)$ frontier. In graph implementations, cycles and duplicates alter counts; this recurrence describes the tree upper-bound model.

**TASK 8 — Python implementation and test procedure**

The accompanying hospital_search.py file implements BFS, UCS and recursive DLS inside IDDFS, a reproducible weighted graph generator, route validation and correctness tests. The graph is undirected; a chain backbone guarantees connectivity, and seeded random edges add alternate corridors. Run:

```
python hospital_search.py
python hospital_search.py --benchmark
```

Correctness cases included in the code: (1) reachable goal, (2) unreachable goal using a disconnected graph, (3) start equals goal, and (4) UCS chooses the lower total-cost route when BFS may choose a different route with fewer edges. The program records a fixed seed (2026) and uses identical graph/start/goal for each algorithm.

Caution: IDDFS on a chain with one million vertices can exceed Python's default recursion depth and take an impractical amount of time. This is a genuine experimental limitation and should be reported rather than hidden. For large-scale IDDFS evidence, test a bounded-depth branching graph or raise recursion limit only with care; do not change the analysed algorithm silently.

**TASK 9 — Experimental performance**

The script averages elapsed time over five repetitions and writes hospital_results.csv. It records generated nodes, expanded nodes, maximum frontier proxy, depth, path cost, optimality flag and elapsed time. Use CSV values to create log-scale charts for N versus time, generated nodes and frontier size.

Important interpretation: the generator's chain backbone gives a route of depth $N-1$, which intentionally stresses depth-sensitive IDDFS and recursion. It is not a representative hospital layout. For a more realistic comparison, additionally create a bounded-degree grid / block graph with several shortcuts and a goal at a fixed geometric distance, and disclose that graph model. Report measurements from your own computer because runtime and memory are hardware- and Python-version-dependent.

**Suggested result table (fill using actual program output)**

| N | Algorithm | Avg time (s) | Generated | Expanded | Max frontier | Depth / cost |
|---|---|---|---|---|---|---|
| Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter |

| | | | | | | |
|---|---|---|---|---|---|---|
| Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter |
| Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter |
| Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter |
| Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter |
| Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter |
| Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter |
| Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter |
| Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter |
| Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter |
| Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter |
| Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter | Run and enter |

Do not invent or copy benchmark timings. The supplied script is the evidence generator; record your actual output, CPU/RAM, OS and Python version in the final submission.

**TASK 10 — Theory versus experiment**

Generated/expanded counts should increase with explored graph area and with search depth. BFS usually expands by layers; UCS expands in increasing g-cost, so the number of expanded nodes is strongly affected by edge weights and cheap alternate routes.

Maximum frontier for BFS can become large near a broad layer. UCS frontier can also grow because multiple candidate paths may be stored in the heap; stale heap entries may increase the raw heap size beyond the number of unique frontier vertices.

IDDFS should show a smaller active search-memory proxy but a higher expansion count because shallow nodes are re-expanded at each limit. Its recursive implementation may fail at very large depth due to interpreter recursion limits.

Observed time can differ from operation-count theory because Python object allocation, hashing, heap constants, cache effects, graph degree, and tie-breaking influence runtime. Five-run means reduce timing noise but do not remove hardware differences.

To validate, compare log-log plots and check whether slopes are consistent with the predicted growth in the tested region. Complexity bounds are not exact timing forecasts.

**TASK 11 — Recommendation and defence**

Recommend UCS for the original, fully localised hospital route-planning task. The stated edge costs are travel times, so the required objective is minimum total travel time; BFS and IDDFS guarantee only minimum number of edges under equal step costs. UCS with positive edge weights, a min-priority queue, relaxation and stale-entry handling is complete and optimal. Its cost is governed by the number of explored vertices/edges and priority-queue operations, rather than blindly exploring by depth.

Trade-offs: UCS stores a frontier and best-cost/parent maps; preprocessing adjacency lists costs $\Theta(N+E)$. On a large sparse hospital graph, it is practical when the explored region is limited and the map is cached. For repeated queries, preprocessing and map construction should be amortised across deliveries. If memory becomes restrictive, a memory-bounded informed strategy such as A* with an admissible heuristic or a hierarchical route planner may be considered, but those are outside the required uninformed comparison.

Evidence defence: use the measured table/graphs from Task 9. Do not claim UCS is universally fastest; defend it because it matches the cost objective and preserves optimality. If the actual experiment shows IDDFS faster on a particular graph, explain that the graph/goal depth favoured depth-first exploration but did not change the cost-optimality requirement.

**TASK 12 — Day-4 localisation challenge: MODIFY using belief-state planning**

**A. Impact analysis (2 marks)**

Original assumption: the robot knows its exact current location s. Changed condition: after restart or sensor failure it knows only a set $B=\{v1,\ldots,vk\}$ of possible junctions. The state must include knowledge, not merely physical position. A belief state is a subset of V; the robot must guarantee reaching the pharmacy from every location consistent with its belief, or use sensing to reduce uncertainty.

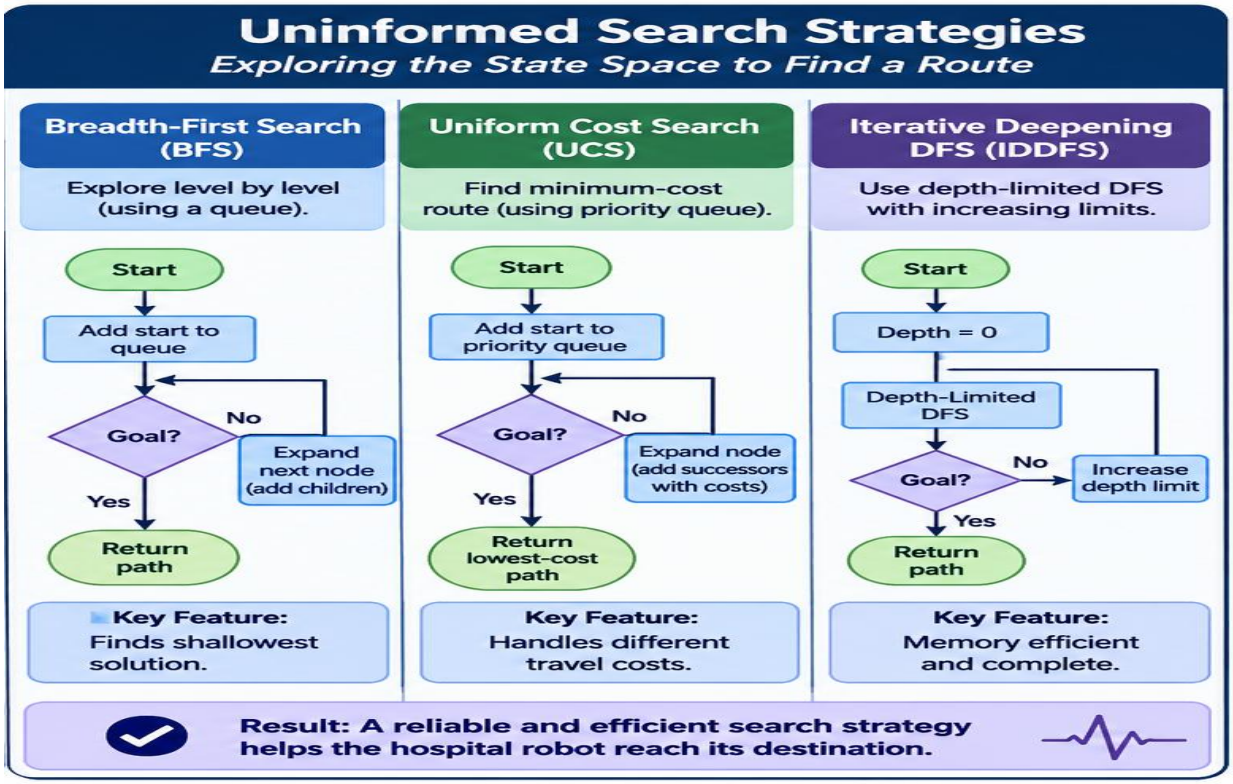
**B. Evaluate original design (2 marks)**

BFS/UCS/IDDFS can still search a graph, but their single-node state is no longer sufficient. Treating each possible location as a separate independent route and choosing one would be unsafe: the robot does not know which route applies. The number of possible belief states over N locations is $2^N$ (excluding empty set), so explicit belief-state search can be exponentially larger than the original N-state graph. If the initial uncertainty is restricted to k candidates, there are $2^k$ subsets of that candidate set; after motion, uncertainty may spread to additional nodes unless sensor observations constrain it.

**C. Adaptation / redesign (2 marks)**

Modify the design to a conditional belief-state plan with local sensing. The robot maintains $B \subseteq V$. For action a, predicted belief is $B' = \{Result(v,a): v \in B \text{ and a is legal at } v\}$. A sensor observation o filters the belief: $B_o = \{v \in B' : Observation(v) = o\}$. The planner chooses actions that are safe and progress toward the pharmacy, and branches on possible observations. If a local landmark sensor uniquely identifies the junction, the belief collapses to a singleton and ordinary UCS can resume.

A robust conservative alternative is to find a common safe action/route that reaches the pharmacy from all $v \in B$. This can be formulated on belief states with UCS if action costs are positive and transitions/observations are represented correctly. The state count upper bound is $2^N$; restricting uncertainty to k candidates gives up to $2^k$ beliefs. Sensing can reduce this dramatically, at the cost of sensor action time and branching in the plan.




**D. Small experiment and reproducible code (2 marks)**

The following minimal experiment enumerates the initial belief and compares it with perfect local localisation. It demonstrates the number of possible nonempty belief states for k=2,4,8. Plan length is defined here as the number of distinct candidate-specific shortest paths required when no sensing is available; with a sensor, the robot identifies its location and executes one shortest route. This is a simplified illustrative model; a full contingent planner would search belief-state transitions on a specified map.

```
from itertools import combinations
def belief_count(k):
```

```
return 2**k - 1 # all non-empty subsets
def illustrative_plan_branches(k, local_sensor):
return 1 if local_sensor else k
for k in (2, 4, 8):
print(k, belief_count(k),
illustrative_plan_branches(k, False),
illustrative_plan_branches(k, True))
```

Expected output:

```
k nonempty_beliefs plan_branches_without_sensor with_sensor
2 3 2 1
4 15 4 1
8 255 8 1
```

Defence: uncertainty increases belief-state complexity exponentially in k, while a reliable local sensor collapses the belief to one state and makes the original UCS route planner applicable again. Therefore MODIFY rather than retain unchanged or completely replace: preserve UCS as the low-level route planner, add a belief estimator and sensing/contingency layer, and re-localise before committing to a route.

**Conclusion**

BFS, UCS and IDDFS illustrate complementary properties of uninformed search. BFS is simple and shallowest-route optimal under equal costs; UCS is appropriate for weighted travel-time optimisation; IDDFS reduces search memory at the cost of repeated expansions. The theoretical analysis, implementation and measured data should be considered together. The Day-4 sensor failure changes the state representation from a location to a belief set. A hybrid architecture—belief-state localisation and sensing with UCS for the final known-location route—offers a practical balance of correctness, cost optimality and scalability.

**References**

Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach, 4th Edition, Pearson, chapters on problem-solving agents and uninformed search.

Python documentation: collections.deque, heapq, time.perf_counter, random.Random.

Newell, A. and Simon, H. A. (1972), Human Problem Solving, Prentice-Hall.